

中山大学电子与通信工程学院

《数字信号处理实验》实验报告

姓名：王晨晖 学号：24307163 专业：电子信息工程

综合实验 1 语音信号处理

一、实验目的

1. 掌握 Matlab 的基本编程语句；
2. 掌握在 Window 环境下音频信号的采集、读取与播放方法；
3. 掌握数字信号处理的基本概念、基本理论和基本方法；
4. 掌握 Matlab 处理数字信号、进行频谱分析、设计数字滤波器的编程方法。

二、实验要求

1. 每个实验内容都要清晰展示实验过程和实验结果。
2. 绘图时必须明确标出各个坐标轴的意义和单位。
3. 绘制多个子图时要明确标出各个子图所画的内容。
4. 绘制多个信号时采用不同的线型和颜色区分、添加图例。
5. 严禁抄袭、复制，发现雷同报告或代码双方成绩均作废！！

三、实验原理

- 采样定理：采样频率 F_s 必须大于信号最高频率的 2 倍（Nyquist 频率 $f_n = F_s/2$ ），否则发生频谱混叠，高频分量会折叠到低频区造成失真。
- FFT 频谱分析：通过快速傅里叶变换将时域离散信号变换到频域，揭示信号各频率成分的幅度分布，频谱分辨率 $\Delta f = F_s/N$ 。
- IIR 滤波器：无限冲激响应滤波器，系统函数 $H(z) = B(z)/A(z)$ ，存在反馈回路，可用较低阶数实现锐截止，相位非线性。
- FIR 滤波器：有限冲激响应滤波器， $H(z) = \sum b_k z^{-k}$ ，无反馈、始终稳定、可实现严格线性相位；本实验采用 Kaiser 窗函数法设计。
- 滤波器指标：通带截止频率 f_p 、阻带截止频率 f_s 、通带最大衰减 A_p （通带内容许幅度波动）、阻带最小衰减 A_s （阻带最小抑制量），过渡带为 $|f_s - f_p|$ 。
- 变声原理：不同人群声带振动基频不同（男 80~400Hz、女 150~1100Hz、童 250~1300Hz），通过重采样（resample）改变音调，再用二次重采样恢复原始时长，实现“变调不变速”。

四、实验所用 MATLAB 函数

- `[y, Fs] = audioread(file)` — 读取 WAV/MP3 等音频文件，返回采样数据 y 和采样频率 F_s

- audiowrite(file, y, Fs) — 将音频数据写入 WAV 文件
- sound(y, Fs) — 以指定采样率播放音频
- $Y = \text{fft}(x) / \text{fft}(x, N)$ — 快速傅里叶变换，用于频谱分析
- $[N, Wn] = \text{buttord}(wp, ws, Ap, As)$ — 计算 Butterworth 滤波器最小阶数与 3dB 截止频率
- $[b, a] = \text{butter}(N, Wn, 'type')$ — 设计 Butterworth IIR 滤波器，返回分子 b 和分母 a 系数
- $[N, Wn, beta] = \text{kaiserord}(f, a, dev, Fs)$ — 估算 Kaiser 窗 FIR 滤波器的阶数、截止频率和 beta 参数
- $b = \text{fir1}(N, Wn, 'type', \text{kaiser}(N+1, beta))$ — 基于 Kaiser 窗函数法设计 FIR 滤波器
- $[H, w] = \text{freqz}(b, a, N, Fs)$ — 计算滤波器频率响应（幅度与相位），用于绘制幅频/相频特性曲线
- $y = \text{filter}(b, a, x)$ — 基于差分方程对信号 x 进行 IIR/FIR 滤波
- $y = \text{resample}(x, p, q)$ — 以 p/q 倍率重采样（含抗混叠滤波），用于语音变声中改变采样率

五、实验内容、方法、代码与结果

题目 1：音频信号处理

1. 要求利用 Windows 下的录音机，录制一段自己的语音，时间在 5 秒内，存 *.wav 的文件（或者用题目提供的 .wav 文件，文件名为，可以截取部分）。然后在 Matlab 软件平台下，用函数 audioread 读出语音信号（可以只读出一个声道），记住采样率和采样点数。

2. 对语音信号进行快速傅里叶变换，得到信号的幅频特性，分析声音的幅频特性。

3. 设计数字滤波器，各种滤波器的性能指标：

(1) 低通滤波器性能指标 $f_p=500\text{Hz}$, $f_s=800\text{Hz}$, $A_s=20\text{dB}$, $A_p=1\text{dB}$ 。

(2) 高通滤波器性能指标 $f_s=1200\text{Hz}$, $f_p=2000\text{Hz}$, $A_s=20\text{dB}$, $A_p=1\text{dB}$ 。

(3) 带通滤波器性能指标 $f_{p1}=1200\text{Hz}$, $f_{pu}=3000\text{Hz}$, $f_{s1}=1000\text{Hz}$, $f_{su}=3200\text{Hz}$, $A_s=15\text{dB}$, $A_p=1\text{dB}$ 。

根据上述要求，分别设计 IIR 滤波器和 FIR 滤波器（用函数），分别对信号进行滤波（filter 函数），要求画出滤波器幅频响应和滤波前后（时域画在一张图内，频域画在一张图内）的时域波形及幅频。

4. 分别采集成年男、女和童声三种声音之语音的相互语音程序（若为男同学，只需采集男女之语音，男和男童之语音的相互语音；若为女同学，只需采集男女之语音，女和女童之语音的相互语音），观察语音信号的波形及幅频，要求将语音后的语音信号保存为 WAV 格式（作成的包要包含 3 种 WAV 文件：男声、女声和童声）。提示，男声的语音更低，基频一般在 80-400Hz，女声在 150-1100Hz，童声在 250-1300Hz。

解答 1：

```
%insert code here
[filename, pathname] = uigetfile('*.wma', '请选择音频文件');
if isequal(filename, 0)
    disp('用户取消选择');
    return;
end

[audio, fs] = audioread(fullfile(pathname, filename));

% 转为单声道处理
```

```

if size(audio, 2) == 2
    audio = audio(:, 1);
end

% 截取 20 秒片段（从第 30 秒开始）
time_pro = 20;
time_shift = 30;
audio = audio(time_shift*fs+1 : (time_shift+time_pro)*fs, 1);

N = length(audio);           % 采样点数
t = (0:N-1) / fs;           % 时间轴
fprintf('采样频率: %d Hz\n', fs);

```

采样频率: 44100 Hz

```
fprintf('采样点数: %d\n', N);
```

采样点数: 882000

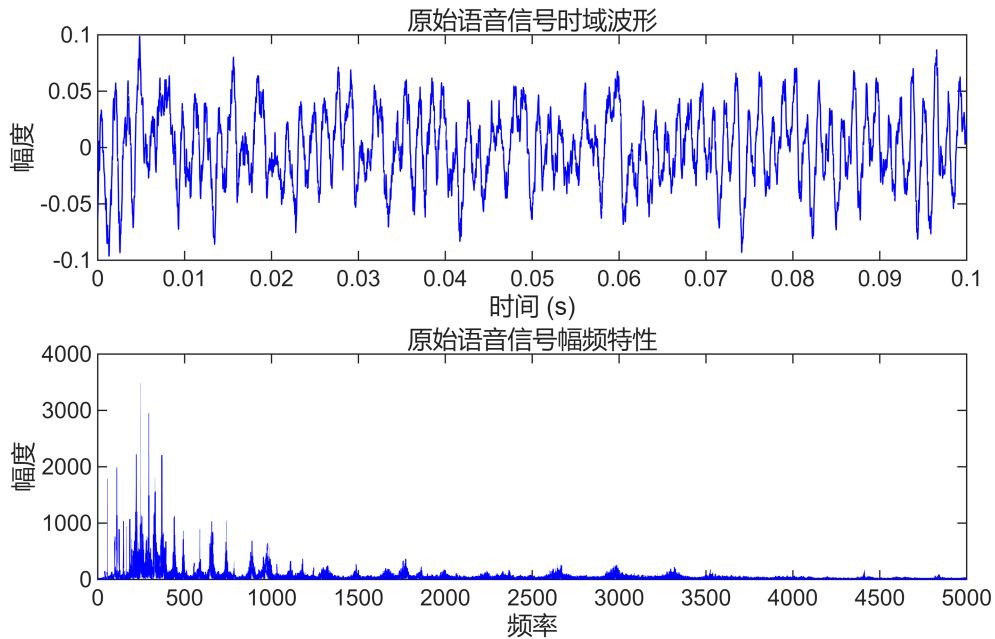
```

X = fft(audio);               % 快速傅里叶变换
f_axis = (0:N-1) * (fs/N);    % 频率轴
mag_X = abs(X);               % 幅度谱
half = floor(N/2);            % 单边谱截断点

figure;
subplot(2,1,1);
plot(t, audio, 'b');
title('原始语音信号时域波形');
xlabel('时间 (s)'); ylabel('幅度');
xlim([0 0.1]);

subplot(2,1,2);
plot(f_axis(1:half), mag_X(1:half), 'b');
title('原始语音信号幅频特性');
xlabel('频率'); ylabel('幅度');
xlim([0 5000]);

```



```
% ----- (1) 低通滤波器 -----
fp_lp = 500; fst_lp = 800; Ap_lp = 1; As_lp = 20;

% --- IIR 椭圆滤波器 ---
wp_lp = fp_lp / (fs/2);
ws_lp = fst_lp / (fs/2);
[n_iir_lp, Wn_iir_lp] = ellipord(wp_lp, ws_lp, Ap_lp, As_lp);
[b_iir_lp, a_iir_lp] = ellip(n_iir_lp, Ap_lp, As_lp, Wn_iir_lp, 'low');

% --- FIR Kaiser 窗滤波器 ---
dev_p_lp = 10^(Ap_lp/20) - 1;          % 通带偏差
dev_s_lp = 10^(-As_lp/20);             % 阻带偏差
[n_fir_lp, Wn_fir_lp, beta_lp] = kaiserord( ...
    [fp_lp, fst_lp], [1 0], [dev_p_lp, dev_s_lp], fs);
b_fir_lp = fir1(n_fir_lp, Wn_fir_lp, 'low', ...
    kaiser(n_fir_lp+1, beta_lp), 'noscale');

% 滤波
y_fir_lp = filter(b_fir_lp, 1, audio);
y_iir_lp = filter(b_iir_lp, a_iir_lp, audio);
Y_fir_lp = abs(fft(y_fir_lp));
Y_iir_lp = abs(fft(y_iir_lp));

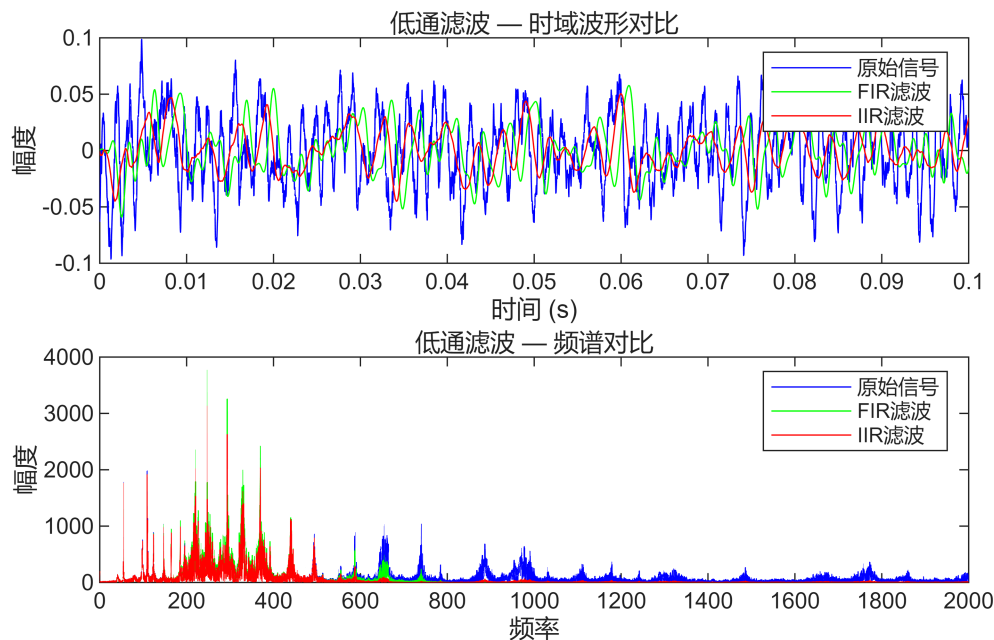
% 低通 - 时域+频域对比 (figure 1)
figure;
subplot(2,1,1);
plot(t, audio, 'b'); hold on;
plot(t, y_fir_lp, 'g');
plot(t, y_iir_lp, 'r');
```

```

title('低通滤波 - 时域波形对比');
xlabel('时间 (s)'); ylabel('幅度');
legend('原始信号', 'FIR 滤波', 'IIR 滤波'); xlim([0 0.1]);

subplot(2,1,2);
plot(f_axis(1:half), mag_X(1:half), 'b'); hold on;
plot(f_axis(1:half), Y_fir_lp(1:half), 'g');
plot(f_axis(1:half), Y_iir_lp(1:half), 'r');
title('低通滤波 - 频谱对比');
xlabel('频率'); ylabel('幅度');
legend('原始信号', 'FIR 滤波', 'IIR 滤波'); xlim([0 2000]);

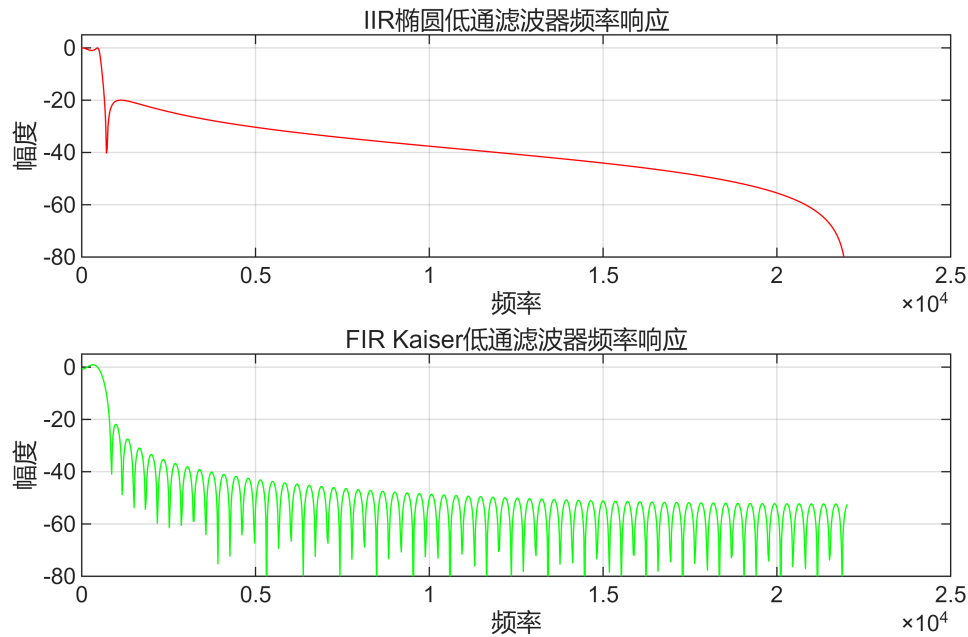
```



```

% 低通 - 滤波器频率响应 (figure 2)
figure;
[H_iir_lp, w_lp] = freqz(b_iir_lp, a_iir_lp, 1024, fs);
[H_fir_lp, ~] = freqz(b_fir_lp, 1, 1024, fs);
subplot(2,1,1);
plot(w_lp, 20*log10(abs(H_iir_lp)), 'r');
title('IIR 椭圆低通滤波器频率响应');
xlabel('频率'); ylabel('幅度'); ylim([-80 5]); grid on;
subplot(2,1,2);
plot(w_lp, 20*log10(abs(H_fir_lp)), 'g');
title('FIR Kaiser 低通滤波器频率响应');
xlabel('频率'); ylabel('幅度'); ylim([-80 5]); grid on;

```



```
% ----- (2) 高通滤波器 -----
fst_hp = 1200; fp_hp = 2000; Ap_hp = 1; As_hp = 20;

% --- IIR ---
wp_hp = fp_hp / (fs/2);
ws_hp = fst_hp / (fs/2);
[n_iir_hp, Wn_iir_hp] = ellipord(wp_hp, ws_hp, Ap_hp, As_hp);
[b_iir_hp, a_iir_hp] = ellip(n_iir_hp, Ap_hp, As_hp, Wn_iir_hp, 'high');

% --- FIR ---
dev_p_hp = 10^(Ap_hp/20) - 1;
dev_s_hp = 10^(-As_hp/20);
[n_fir_hp, Wn_fir_hp, beta_hp] = kaiserord( ...
    [fst_hp, fp_hp], [0 1], [dev_s_hp, dev_p_hp], fs);
if mod(n_fir_hp, 2) == 1 % 高通需要偶数阶
    n_fir_hp = n_fir_hp + 1;
end
b_fir_hp = fir1(n_fir_hp, Wn_fir_hp, 'high', ...
    kaiser(n_fir_hp+1, beta_hp), 'noscale');

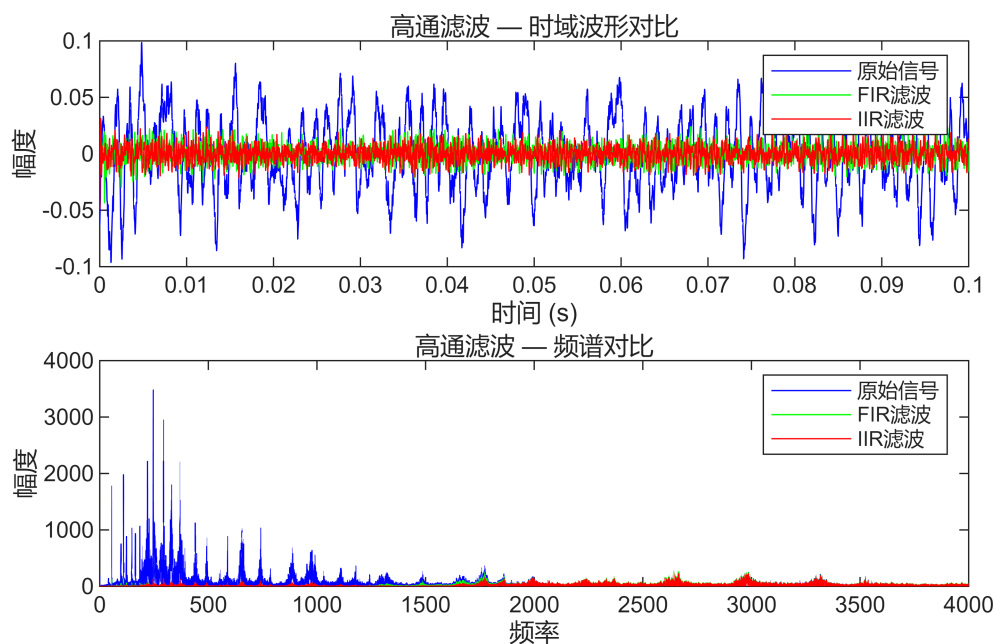
% 滤波
y_fir_hp = filter(b_fir_hp, 1, audio);
y_iir_hp = filter(b_iir_hp, a_iir_hp, audio);
Y_fir_hp = abs(fft(y_fir_hp));
Y_iir_hp = abs(fft(y_iir_hp));

% 高通 - 时域+频域对比
figure;
subplot(2,1,1);
```

```

plot(t, audio, 'b'); hold on;
plot(t, y_fir_hp, 'g');
plot(t, y_iir_hp, 'r');
title('高通滤波 - 时域波形对比');
xlabel('时间 (s)'); ylabel('幅度');
legend('原始信号', 'FIR 滤波', 'IIR 滤波'); xlim([0 0.1]);
subplot(2,1,2);
plot(f_axis(1:half), mag_X(1:half), 'b'); hold on;
plot(f_axis(1:half), Y_fir_hp(1:half), 'g');
plot(f_axis(1:half), Y_iir_hp(1:half), 'r');
title('高通滤波 - 频谱对比');
xlabel('频率'); ylabel('幅度');
legend('原始信号', 'FIR 滤波', 'IIR 滤波'); xlim([0 4000]);

```

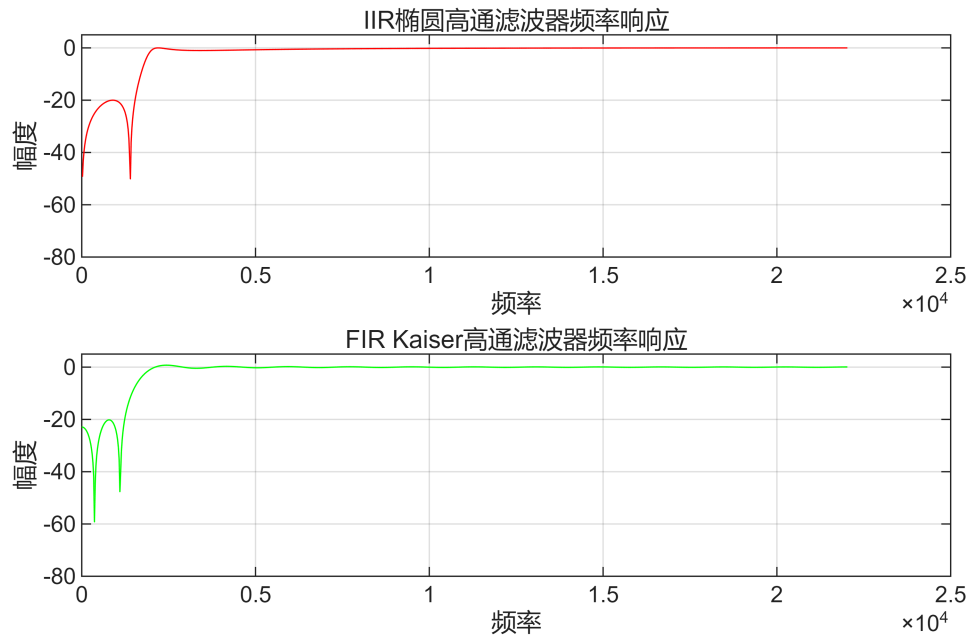


% 高通 - 滤波器频率响应

```

figure;
[H_iir_hp, w_hp] = freqz(b_iir_hp, a_iir_hp, 1024, fs);
[H_fir_hp, ~] = freqz(b_fir_hp, 1, 1024, fs);
subplot(2,1,1);
plot(w_hp, 20*log10(abs(H_iir_hp)), 'r');
title('IIR 椭圆高通滤波器频率响应');
xlabel('频率'); ylabel('幅度'); ylim([-80 5]); grid on;
subplot(2,1,2);
plot(w_hp, 20*log10(abs(H_fir_hp)), 'g');
title('FIR Kaiser 高通滤波器频率响应');
xlabel('频率'); ylabel('幅度'); ylim([-80 5]); grid on;

```



```
% ----- (3) 带通滤波器 -----
fsl_bp = 1000; fpl_bp = 1200; fpu_bp = 3000; fsu_bp = 3200;
Ap_bp = 1; As_bp = 15;

% --- IIR ---
wp_bp = [fpl_bp, fpu_bp] / (fs/2);
ws_bp = [fsl_bp, fsu_bp] / (fs/2);
[n_iir_bp, Wn_iir_bp] = ellipord(wp_bp, ws_bp, Ap_bp, As_bp);
[b_iir_bp, a_iir_bp] = ellip(n_iir_bp, Ap_bp, As_bp, Wn_iir_bp, 'bandpass');

% --- FIR ---
dev_p_bp = 10^(Ap_bp/20) - 1;
dev_s_bp = 10^(-As_bp/20);
[n_fir_bp, Wn_fir_bp, beta_bp] = kaiserord( ...
    [fsl_bp, fpl_bp, fpu_bp, fsu_bp], [0 1 0], ...
    [dev_s_bp, dev_p_bp, dev_s_bp], fs);
b_fir_bp = fir1(n_fir_bp, Wn_fir_bp, 'bandpass', ...
    kaiser(n_fir_bp+1, beta_bp), 'noscale');

% 滤波
y_fir_bp = filter(b_fir_bp, 1, audio);
y_iir_bp = filter(b_iir_bp, a_iir_bp, audio);
Y_fir_bp = abs(fft(y_fir_bp));
Y_iir_bp = abs(fft(y_iir_bp));

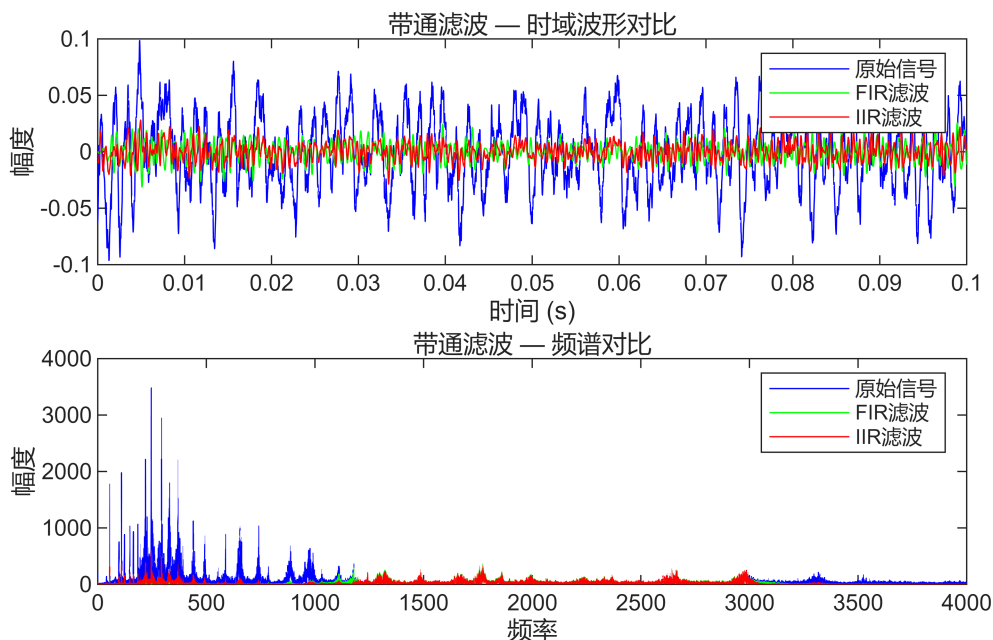
% 带通 - 时域+频域对比
figure;
subplot(2,1,1);
plot(t, audio, 'b'); hold on;
```



```

plot(t, y_fir_bp, 'g');
plot(t, y_iir_bp, 'r');
title('带通滤波 - 时域波形对比');
xlabel('时间 (s)'); ylabel('幅度');
legend('原始信号', 'FIR 滤波', 'IIR 滤波'); xlim([0 0.1]);
subplot(2,1,2);
plot(f_axis(1:half), mag_X(1:half), 'b'); hold on;
plot(f_axis(1:half), Y_fir_bp(1:half), 'g');
plot(f_axis(1:half), Y_iir_bp(1:half), 'r');
title('带通滤波 - 频谱对比');
xlabel('频率'); ylabel('幅度');
legend('原始信号', 'FIR 滤波', 'IIR 滤波'); xlim([0 4000]);

```

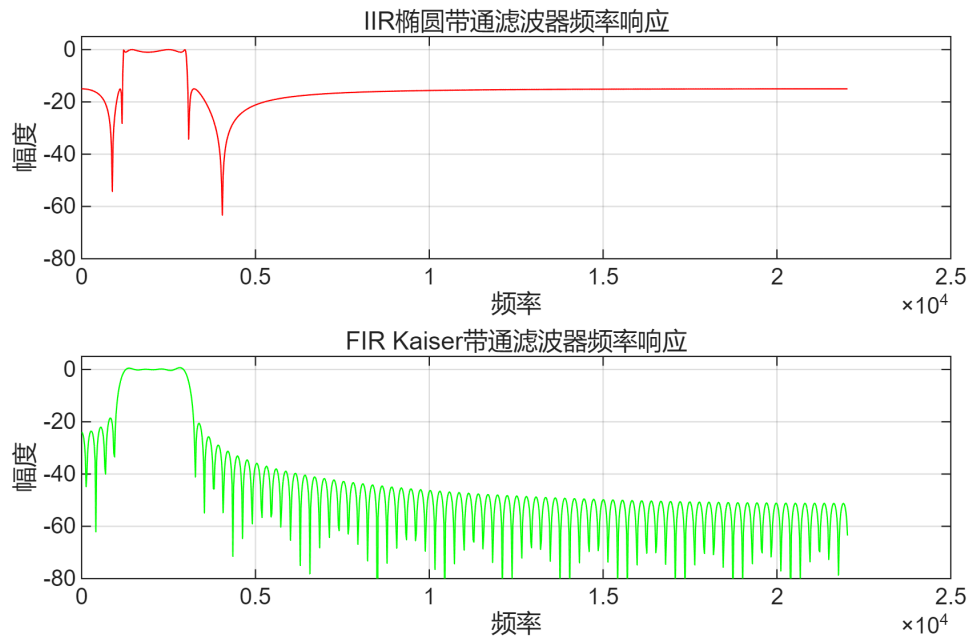


% 带通 - 滤波器频率响应

```

figure;
[H_iir_bp, w_bp] = freqz(b_iir_bp, a_iir_bp, 1024, fs);
[H_fir_bp, ~] = freqz(b_fir_bp, 1, 1024, fs);
subplot(2,1,1);
plot(w_bp, 20*log10(abs(H_iir_bp)), 'r');
title('IIR 椭圆带通滤波器频率响应');
xlabel('频率'); ylabel('幅度'); ylim([-80 5]); grid on;
subplot(2,1,2);
plot(w_bp, 20*log10(abs(H_fir_bp)), 'g');
title('FIR Kaiser 带通滤波器频率响应');
xlabel('频率'); ylabel('幅度'); ylim([-80 5]); grid on;

```



%变声处理

```
disp('==== 变声处理 =====');
```

==== 变声处理 =====

```
disp('提示：男声基频 80-400Hz，女声 150-1100Hz，童声 250-1300Hz');
```

提示：男声基频 80-400Hz，女声 150-1100Hz，童声 250-1300Hz

```
disp('变声方法：STFT + imresize 时间轴缩放 + ISTFT + resample 恢复时长');
```

变声方法：STFT + imresize 时间轴缩放 + ISTFT + resample 恢复时长

% 选择原始音频类型

```
fprintf('\n 请选择原始音频的类型：\n');
```

请选择原始音频的类型：

```
fprintf(' 1 - 男声（默认）\n');
```

1 - 男声（默认）

```
fprintf(' 2 - 女声\n');
```

2 - 女声

```
fprintf(' 3 - 童声\n');
```

3 - 童声

```
choice = input('输入选择 (1/2/3): ');
```

```
switch choice
```

```

case 2
    original_type = 'female';
    fprintf('原始音频类型：女声\n');
case 3
    original_type = 'child';
    fprintf('原始音频类型：童声\n');
otherwise
    original_type = 'male';
    fprintf('原始音频类型：男声\n');
end

```

原始音频类型：女声

```

audio_norm = audio / max(abs(audio));

% 根据原始类型进行变声
switch original_type
case 'male'
    female_voice = vc_imresize(audio_norm, fs, 1.8, '男声→女声');
    child_voice = vc_imresize(audio_norm, fs, 2.2, '男声→童声');
    male_voice = audio_norm;
case 'female'
    male_voice = vc_imresize(audio_norm, fs, 0.55, '女声→男声');
    child_voice = vc_imresize(audio_norm, fs, 2.2/1.8, '女声→童声');
    female_voice = audio_norm;
case 'child'
    male_voice = vc_imresize(audio_norm, fs, 0.45, '童声→男声');
    female_voice = vc_imresize(audio_norm, fs, 0.8, '童声→女声');
    child_voice = audio_norm;
end

```

女声→男声：scale=0.55，输出长度=882000
 女声→童声：scale=1.22，输出长度=882000

```

% ===== 保存 =====
audiowrite('male_voice.wav', male_voice, fs);
audiowrite('female_voice.wav', female_voice, fs);
audiowrite('child_voice.wav', child_voice, fs);
fprintf('\n 已保存:\n  male_voice.wav\n  female_voice.wav\n  child_voice.wav\n');

```

已保存：
 male_voice.wav
 female_voice.wav
 child_voice.wav

```

% ===== 绘图（时域+频谱+语谱图） =====
figure;

Nm=length(male_voice);  Nf=length(female_voice);  Nc=length(child_voice);

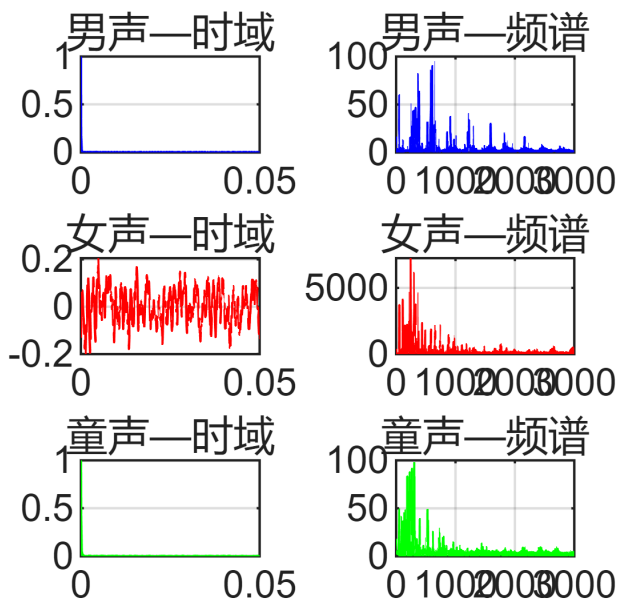
```

% 时域

```
subplot(3,3,1); plot((0:Nm-1)/fs, male_voice, 'b'); title('男声—时域'); xlim([0
.05]); grid on;
subplot(3,3,4); plot((0:Nf-1)/fs, female_voice, 'r'); title('女声—时域'); xlim([0
.05]); grid on;
subplot(3,3,7); plot((0:Nc-1)/fs, child_voice, 'g'); title('童声—时域'); xlim([0
.05]); grid on;
```

% 频谱

```
Fm=abs(fft(male_voice)); Ff=abs(fft(female_voice)); Fc=abs(fft(child_voice));
fm=(0:floor(Nm/2))*(fs/Nm); ff=(0:floor(Nf/2))*(fs/Nf); fc=(0:floor(Nc/2))*(fs/
Nc);
subplot(3,3,2); plot(fm(1:min(end,3000*Nm/fs)),Fm(1:min(end,3000*Nm/fs)), 'b');
title('男声—频谱'); xlim([0 3000]); grid on;
subplot(3,3,5); plot(ff(1:min(end,3000*Nf/fs)),Ff(1:min(end,3000*Nf/fs)), 'r');
title('女声—频谱'); xlim([0 3000]); grid on;
subplot(3,3,8); plot(fc(1:min(end,3000*Nc/fs)),Fc(1:min(end,3000*Nc/fs)), 'g');
title('童声—频谱'); xlim([0 3000]); grid on;
```



% ===== 播放菜单 =====

```
fprintf('\n[1]男声 [2]女声 [3]童声 [4]全部 [0]退出\n');
```

```
[1]男声 [2]女声 [3]童声 [4]全部 [0]退出
```

```
play_choice = input('选择播放: ');
while ~isempty(play_choice) && play_choice ~= 0
    switch play_choice
        case 1, soundsc(male_voice,fs); pause(length(male_voice)/fs+.5);
        case 2, soundsc(female_voice,fs); pause(length(female_voice)/fs+.5);
        case 3, soundsc(child_voice,fs); pause(length(child_voice)/fs+.5);
        case 4
```

```

        soundsc(male_voice,fs); pause(length(male_voice)/fs+1);
        soundsc(female_voice,fs); pause(length(female_voice)/fs+1);
        soundsc(child_voice,fs); pause(length(child_voice)/fs+1);
    end
    play_choice = input('继续播放(0/1/2/3/4): ');
end
disp('==== 变声处理完成 ====');
```

==== 变声处理完成 =====

```

%% ===== 变声核心函数 (imresize 方案) =====
function y = vc_imresize(x, fs, scale, label)
    N = length(x);
    wlen = 1024; hop = wlen/4; nfft = wlen;
    win = hanning(wlen);

    % STFT
    [S,~,~] = spectrogram(x, win, wlen-hop, nfft, fs);
    [nf, nt] = size(S);

    % 对复频谱沿时间轴 imresize
    new_nt = max(round(nt / scale), 1);
    S_real = imresize(real(S), [nf, new_nt], 'bilinear');
    S_imag = imresize(imag(S), [nf, new_nt], 'bilinear');
    S_resized = S_real + 1i * S_imag;

    % ISTFT 重构
    y_tmp = istft_simple(S_resized, win, hop, nfft);

    % ===== 修复: 用 interp1 替代 resample =====
    t_old = linspace(0, 1, length(y_tmp))';
    t_new = linspace(0, 1, N)';
    y = interp1(t_old, y_tmp, t_new, 'linear', 0)';
    y = y(:) / max(abs(y(:)));

    fprintf('  %s: scale=%.2f, 输出长度=%d\n', label, scale, length(y));
end
```

```

%% ===== ISTFT 辅助函数 =====
function y = istft_simple(S, win, hop, nfft)
    win = win(:);
    nh = round(hop);
    nf = size(S,2);
    ylen = (nf-1)*nh + nfft;
    y = zeros(ylen,1);
    wsum = zeros(ylen,1);
    for k = 1:nf
        idx = (k-1)*nh + (1:nfft);
        frame = real(ifft(S(:,k), nfft));
```

```

        y(idx) = y(idx) + frame .* win;
        wsum(idx) = wsum(idx) + win.^2;
    end
    wsum(wsum < 1e-10) = 1;
    y = y ./ wsum;
end

```

题目 2：回声消除处理

音信号中的回声现象，本子上室再某一段音信号中叠加了一串延迟且幅度衰减的原声脉冲。假设原始音信号 $x(t)$ ，回声信号相原始音信号的延迟分 t_1 和 t_2 ，幅度衰减系数分 α_1, α_2 ，混有回声的信号 $y(t)$ 可表示为：

$$y(t) = x(t) + \alpha_1 x(t - t_1) + \alpha_2 x(t - t_2)$$

假设通过数字滤波器实现回声消除， $x(t)$ 、 $y(t)$ 的抽样结果 $x(n)$ 、 $y(n)$ ，若抽样率 f_s ，有：

$$y(n) = x(n) + \alpha_1 x(n - f_s t_1) + \alpha_2 x(n - f_s t_2)$$

利用 Windows 下的录音机录制一段音信号（文件格式为 wav 格式）或取音文件 guitar_tune.wav，

- (1) 编制 MATLAB 程序，绘制音信号的时域波形图；
- (2) 对原始音信号进行延迟（注意延迟点数 f_s 和延迟的乘积，可能要取整）得到回声信号，延迟 $t_1 = 1s$ 、 $t_2 = 2s$ ，幅度衰减系数 $\alpha_1 = 0.26$ 、 $\alpha_2 = 0.37$ ，并将回声信号与元音信号叠加，得到混有回声的音信号，绘制信号的音图；
- (3) 设计一个系数函数 $H(z)$ 对混有回声的音信号进行滤波消除回声，绘制消除回声信号的音图，并写出滤波器系数函数表达式。

解答 2：

```

% Insert your code here
fs = 8000;           % 采样频率 (Hz)
tau1 = 0.02;         % 回声 1 延迟时间 (秒)
tau2 = 0.05;         % 回声 2 延迟时间 (秒)
alpha1 = 0.6;        % 回声 1 衰减系数
alpha2 = 0.3;        % 回声 2 衰减系数

K1 = round(tau1 * fs); % 延迟点数 1
K2 = round(tau2 * fs); % 延迟点数 2

fprintf('延迟点数: K1=%d, K2=%d\n', K1, K2);

```

延迟点数: K1=160, K2=400

```

try
    [x_orig, fs_file] = audioread('guitar_tune.wav');

```

```

fs = fs_file;
fprintf('已读取 guitartune.wav, 采样率 fs=%d Hz\n', fs);
catch
    fprintf('未找到 guitartune.wav, 生成测试信号...\n');
    t = 0:1/fs:3;
    x_orig = 0.5*sin(2*pi*440*t) + 0.3*sin(2*pi*880*t) + 0.1*randn(length(t),1);
    x_orig = x_orig / max(abs(x_orig));
end

```

已读取 guitartune.wav, 采样率 fs=44100 Hz

```

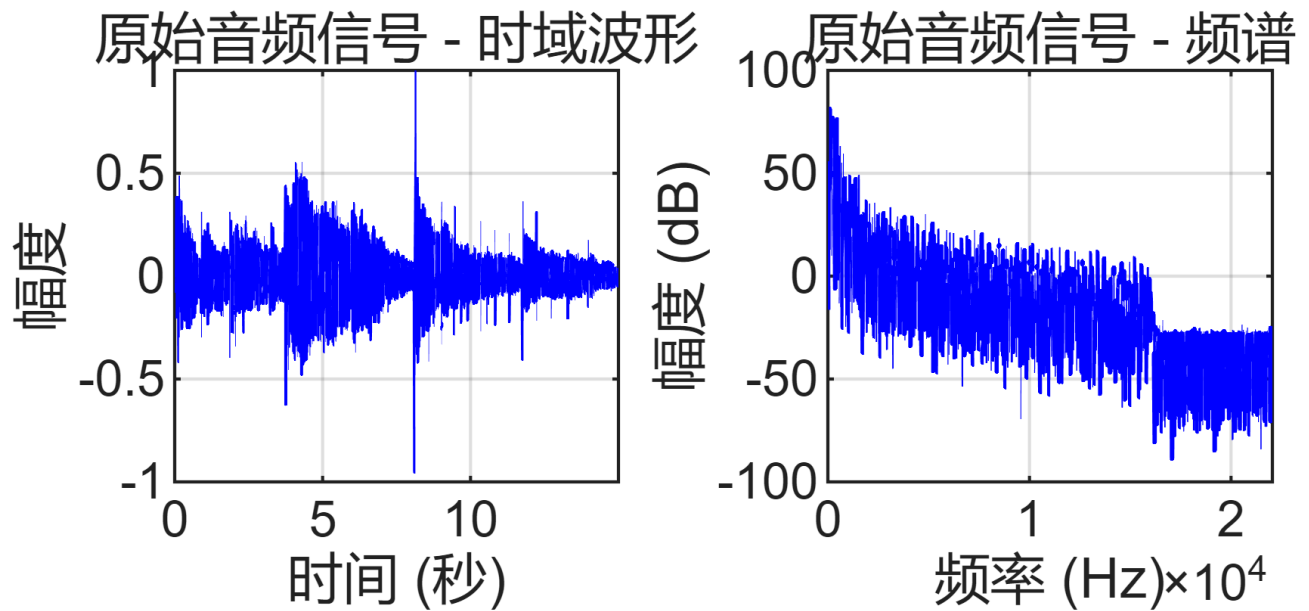
if size(x_orig, 2) == 2
    x_orig = mean(x_orig, 2);
end
x_orig = x_orig(:);
N = length(x_orig);
t_axis = (0:N-1)/fs;

figure('Name', '原始信号分析', 'Position', [100 100 1200 500]);

subplot(1, 2, 1);
plot(t_axis, x_orig, 'b', 'LineWidth', 0.5);
xlabel('时间 (秒)'); ylabel('幅度');
title('原始音频信号 - 时域波形');
grid on; xlim([0 t_axis(end)]);

subplot(1, 2, 2);
NFFT = 2^nextpow2(N);
f_axis = fs*(0:NFFT/2)/NFFT;
X_freq = fft(x_orig, NFFT);
plot(f_axis, 20*log10(abs(X_freq(1:NFFT/2+1)) + eps), 'b', 'LineWidth', 0.8);
xlabel('频率 (Hz)'); ylabel('幅度 (dB)');
title('原始音频信号 - 频谱');
grid on; xlim([0 fs/2]);

```



```

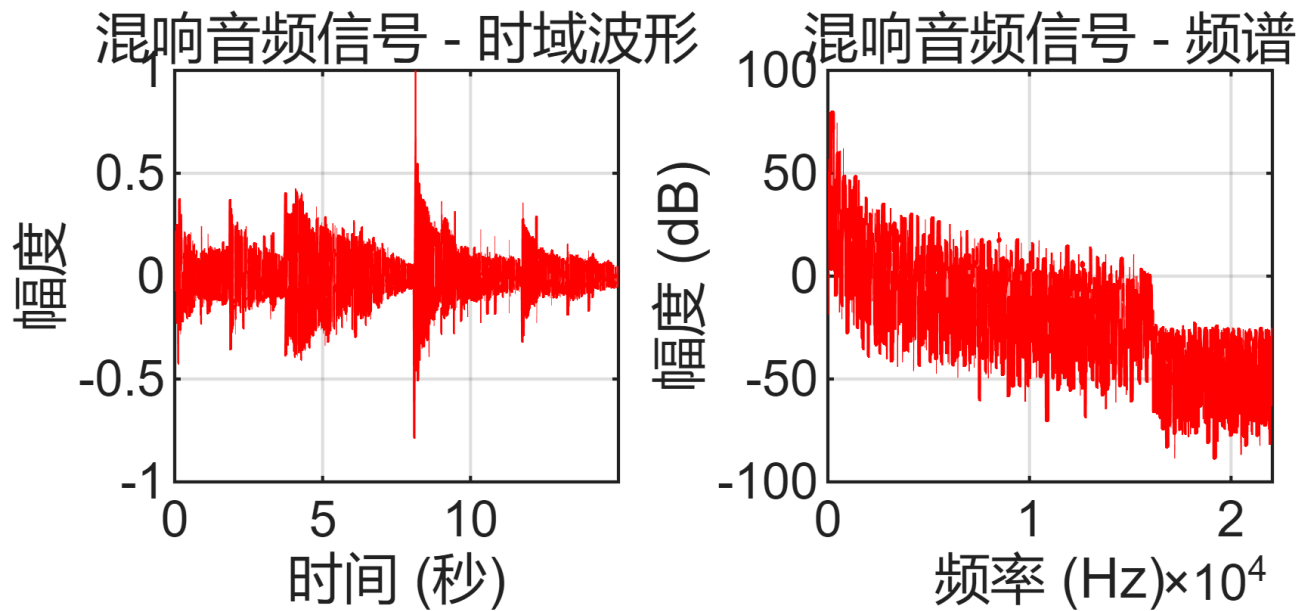
x_delayed1 = [zeros(K1,1); x_orig(1:end-K1)] * alpha1;
x_delayed2 = [zeros(K2,1); x_orig(1:end-K2)] * alpha2;
y = x_orig + x_delayed1 + x_delayed2;
y = y / max(abs(y));

figure('Name', '混响信号分析', 'Position', [100 100 1200 500]);

subplot(1, 2, 1);
plot(t_axis, y, 'r', 'LineWidth', 0.5);
xlabel('时间 (秒)'); ylabel('幅度');
title('混响音频信号 - 时域波形');
grid on; xlim([0 t_axis(end)]);

subplot(1, 2, 2);
Y_freq = fft(y, NFFT);
plot(f_axis, 20*log10(abs(Y_freq(1:NFFT/2+1)) + eps), 'r', 'LineWidth', 0.8);
xlabel('频率 (Hz)'); ylabel('幅度 (dB)');
title('混响音频信号 - 频谱');
grid on; xlim([0 fs/2]);

```

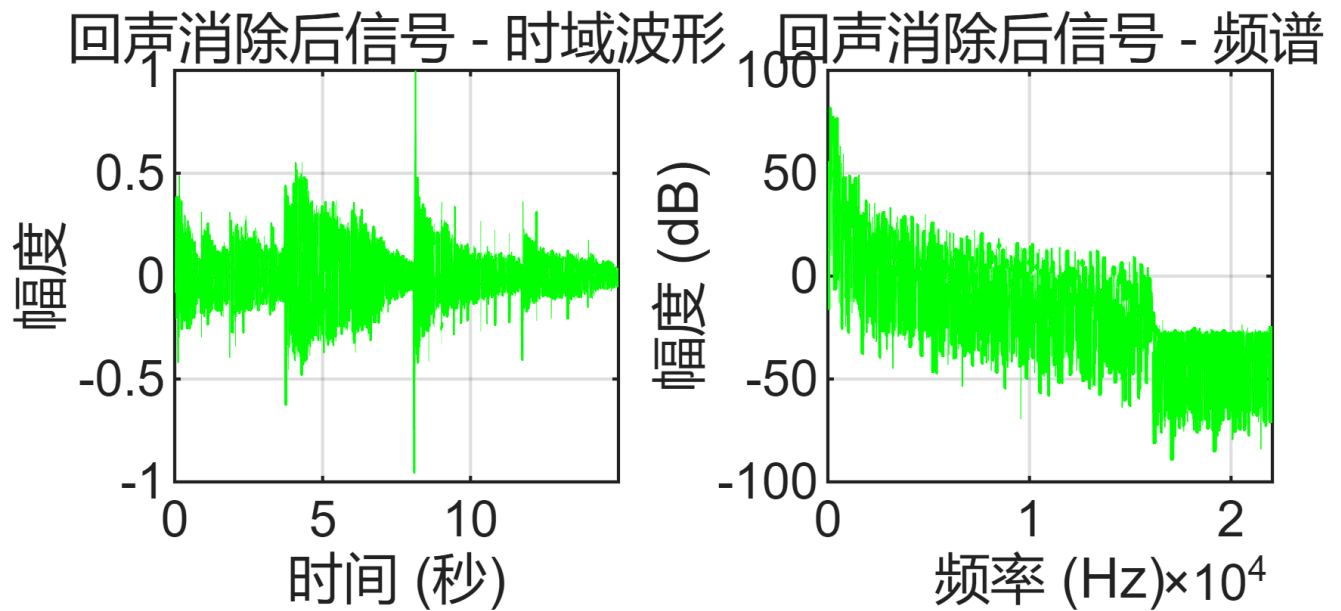



```
% 时域递推:  $\hat{x}[n] = y[n] - \alpha_1 \hat{x}[n-K_1] - \alpha_2 \hat{x}[n-K_2]$ 
x_hat = zeros(N, 1);
for n = 1:N
    x_hat(n) = y(n);
    if n > K1
        x_hat(n) = x_hat(n) - alpha1 * x_hat(n - K1);
    end
    if n > K2
        x_hat(n) = x_hat(n) - alpha2 * x_hat(n - K2);
    end
end
x_hat = x_hat / max(abs(x_hat));

figure('Name', '回声消除结果', 'Position', [100 100 1200 500]);

subplot(1, 2, 1);
plot(t_axis, x_hat, 'g', 'LineWidth', 0.5);
xlabel('时间 (秒)'); ylabel('幅度');
title('回声消除后信号 - 时域波形');
grid on; xlim([0 t_axis(end)]);

subplot(1, 2, 2);
Xhat_freq = fft(x_hat, NFFT);
plot(f_axis, 20*log10(abs(Xhat_freq(1:NFFT/2+1)) + eps), 'g', 'LineWidth', 0.8);
xlabel('频率 (Hz)'); ylabel('幅度 (dB)');
title('回声消除后信号 - 频谱');
grid on; xlim([0 fs/2]);
```



```

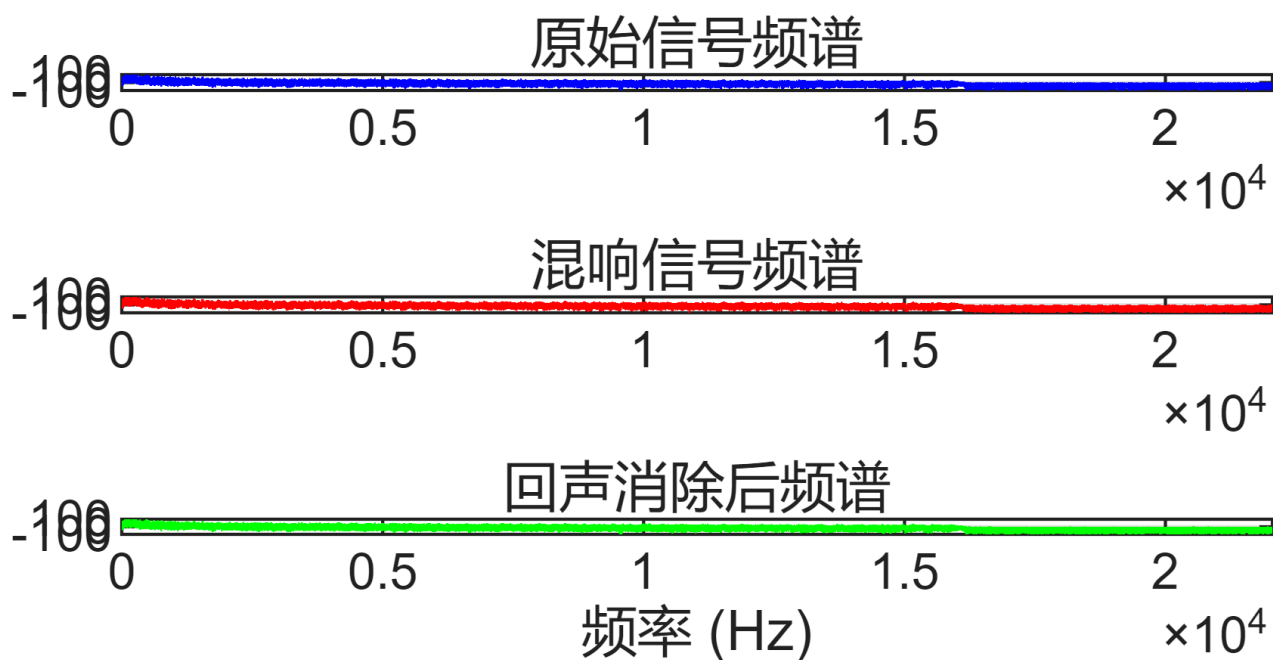
%% 频谱对比图
figure('Name', '频谱对比', 'Position', [100 100 1200 600]);

subplot(3, 1, 1);
plot(f_axis, 20*log10(abs(X_freq(1:NFFT/2+1)) + eps), 'b', 'LineWidth', 0.8);
ylabel('幅度 (dB)'); title('原始信号频谱');
grid on; xlim([0 fs/2]);

subplot(3, 1, 2);
plot(f_axis, 20*log10(abs(Y_freq(1:NFFT/2+1)) + eps), 'r', 'LineWidth', 0.8);
ylabel('幅度 (dB)'); title('混响信号频谱');
grid on; xlim([0 fs/2]);

subplot(3, 1, 3);
plot(f_axis, 20*log10(abs(Xhat_freq(1:NFFT/2+1)) + eps), 'g', 'LineWidth', 0.8);
xlabel('频率 (Hz)'); ylabel('幅度 (dB)'); title('回声消除后频谱');
grid on; xlim([0 fs/2]);

```



```
%% 输出滤波器系统函数
fprintf('\n===== 滤波器系统函数 =====\n');
```

```
===== 滤波器系统函数 =====
```

```
fprintf('回声叠加系统函数  $H(z)$  : \n');
```

回声叠加系统函数 $H(z)$:

```
fprintf('  $H(z) = 1 + 0.3f \cdot z^{-\alpha_1} + 0.3f \cdot z^{-\alpha_2}$ \n', alpha1, K1, alpha2, K2);
```

```
 $H(z) = 1 + 0.600 \cdot z^{-160} + 0.300 \cdot z^{-400}$ 
```

```
fprintf('\n 回声消除逆系统函数  $G(z)$  : \n');
```

回声消除逆系统函数 $G(z)$:

```
fprintf('  $G(z) = 1 / (1 + 0.3f \cdot z^{-\alpha_1} + 0.3f \cdot z^{-\alpha_2})$ \n', alpha1, K1, alpha2, K2);
```

```
 $G(z) = 1 / (1 + 0.600 \cdot z^{-160} + 0.300 \cdot z^{-400})$ 
```

```
fprintf('\n 时域递推公式 : \n');
```

时域递推公式 :

```
fprintf('  $\hat{x}[n] = y[n] - 0.3f \cdot \hat{x}[n-\alpha_1] - 0.3f \cdot \hat{x}[n-\alpha_2]$ \n', alpha1, K1, alpha2, K2);
```

```
 $\hat{x}[n] = y[n] - 0.600 \cdot \hat{x}[n-160] - 0.300 \cdot \hat{x}[n-400]$ 
```

```
%% 保存音频文件
```

```
audiowrite('original.wav', x_orig, fs);  
audiowrite('echo_signal.wav', y, fs);  
audiowrite('echo_cancelled.wav', x_hat, fs);  
fprintf('\n 音频文件已保存 : original.wav, echo_signal.wav, echo_cancelled.wav\n');
```

音频文件已保存 : original.wav, echo_signal.wav, echo_cancelled.wav